
Every problem, algorithm and complexity in your DSA practice repo - explained
in simple words, with the matching LeetCode problem number

How to revise with this booklet

1. Read the **Problem (short)** - the question in one line.
2. Note the **LeetCode number** in the top-right corner, then read the **Question (full)** exactly as it is asked on LeetCode.
3. Read the **Idea in simple words** - the trick that solves it.
4. Follow **How it works** - the steps of the algorithm.
5. Read the **Code** - match every step to its line.
6. Check the coloured **Time / Space** bar at the bottom.

Colours tell you how fast an algorithm is: **green** $O(1)$, **teal** $O(\log n)$, **blue** $O(n)$, **gold** $O(n \log n)$, **red** $O(n^2)$.

Includes: Basic Maths | Bitwise | Patterns | Arrays | Binary Search | Sorting | Strings | Java Collections

Source: DSA practice repository (Java)

Contents

01 Big O in Simple Words	4
02 Basic Maths	6
1. Count the Digits of a Number	8
2. Reverse a Number	9
3. Palindrome Number	10
4. GCD - Euclidean Algorithm	10
03 Bitwise Tricks	13
1. Odd or Even	19
2. Multiply / Divide by 2 with Shifts	19
3. Count Set Bits	20
4. Power of Two	21
5. Appears Once (XOR Superpower)	21
6. Two Unique Numbers	22
04 Patterns	25
1. Pattern Gallery (1-7)	30
05 Arrays	35
1. Reverse an Array	40
2. Check if the Array is Sorted	40
3. Move Zeros to the End	41
4. Segregate Zeros and Ones	42
5. Count Frequency of Each Element	43
6. Palindrome Check (String)	44
7. Two Sum in a Sorted Array	45
8. Max Sum of Subarray of Size k	45
9. Left Rotate by One	46
10. Second Smallest and Second Largest	47
11. Maximum Subarray Sum - Kadane's Algorithm	48
12. Missing Number (Sum Method)	49
13. Missing Number (XOR Method)	50
14. Maximum Consecutive Ones	50
15. Remove Duplicates from a Sorted Array	51
16. Sort 0, 1, 2 - Dutch National Flag	52
17. Union of Two Sorted Arrays	53
18. Longest Subarray with Sum k	54
06 Binary Search on 1D Arrays	57

1. Binary Search	62
2. Lower Bound	63
3. Upper Bound	64
07 Sorting	67
1. Bubble Sort	71
2. Selection Sort	72
3. Insertion Sort	73
08 Strings	76
1. Reverse a String	80
2. Reverse the Vowels	80
09 Java Collections	83
1. List Interface - ArrayList and LinkedList	88
2. Set Interface - HashSet and TreeSet	90
3. Map Interface - HashMap	91
10 Cheat Sheet	93

PART 01

Big O in Simple Words

The 30-second refresher you need before everything else

Complexity measures how fast a program grows when the input grows. We write it as Big-O, like $O(n)$ or $O(n^2)$. It is a rough label, not an exact time. n usually means the number of items in the input.

$O(1)$ = constant: same work no matter the input size (one lookup).

$O(\log n)$ = halving: cut the problem in half each step (binary search).

$O(n)$ = one pass: look at each item once (most simple loops).

$O(n \log n)$ = split and scan: efficient sorting (merge / quick sort).

$O(n^2)$ = nested loops: for each item, loop over the items again (bubble sort).

Space is the same idea, but for extra memory the program needs. In this booklet every problem shows Time first, Space second. If a code looks long but is $O(n)$, it is usually still the good solution.

Complexity	Colour	Meaning
$O(1)$		Best. Constant work. Arrays give instant access.
$O(\log n)$		Great. Halve the input every step (binary search, GCD).
$O(n)$		Good. Single scan. Most of the problems here.
$O(n \log n)$		Fine. The best general sorting can do.
$O(n^2)$		Slow. Nested loops. Only OK for small inputs.

PART 02

Basic Maths

Digit tricks, reversing, palindromes and GCD - warm-up before arrays

T
h
e
s
e

f
o
u
r

b
u
i
l
d

y
o
u
r

l
o
o
p

a
n
d

m
o
d
u
l
a
r

a
r
i
t
h
m
e

t
i
c

i
n
s
t
i
n
c
t
.

T
h
e

k
e
y

i
d
e
a

e
v
e
r
y
w
h
e
r
e
:

n

%

1
0

p
u
l

l
s

t
h
e

l
a
s
t

d
i
g
i
t
,

n

/

1
0

c
h
o
p
s

i
t

o
f
f
.

1. Count the Digits of a Number

GfG

Problem (short): Given a number like 123456, tell how many digits it has (answer 6).

Practice: [GfG - Count digits in a number](#)

Question (full): Count how many digits are in an integer n. Examples: n = 12345 -> 5; n = 0 -> 1; n = -123 -> 3 (the minus sign is ignored).

Idea in simple words

Every time you divide an integer by 10, one digit disappears. Keep dividing and counting until nothing is left.

Two ways appear in the code: a loop that divides by 10 and counts, and the formula $(\text{int})\log_{10}(n) + 1$ which does it in one shot. Both give the same answer.

How it works

- 1. Take the absolute value of n so negatives do not confuse us.
- 2. While $n > 0$: $n = n / 10$ and $\text{count}++$.
- 3. Print count. If n was 0, the answer is 1.

Code

```
import java.util.*;
class Optimal{
    Optimal(int num){
        int ct = (int) (Math.log10(num) + 1);
        System.out.println(ct);
    }
}
public class CountDigits {
    public static void main(String[] args) {

        int num = Math.abs(-123456);
        Optimal o1 = new Optimal(num);
        int count = 0;
        if (num == 0) {
            System.out.println("1");
            return;
        }
        while (num > 0) {
            num = num / 10;
            count++;
        }
        System.out.println(count);
    }
}
```

TIME: $O(\log_{10} n)$

SPACE: $O(1)$

2. Reverse a Number

LeetCode #7

Problem (short): Reverse the digits. 12355 becomes 55321.

LeetCode 7 - Reverse Integer (Medium). [Solve on LeetCode](#)

Question (full): Given a signed 32-bit integer x, return x with its digits reversed. If reversing x causes the value to go outside the signed 32-bit integer range $[-2^{31}, 2^{31} - 1]$, return 0. Examples: 123 -> 321; -123 -> -321; 120 -> 21.

Heads up: The repo's simple loop is the core logic. LeetCode also wants you to guard against overflow and keep the sign - add those when you solve it there.

Idea in simple words

Build the answer from right to left: take the last digit with $\% 10$, push it into the new number by multiplying the new number by 10, then drop the digit with $/ 10$.

$\text{rev} = \text{rev} * 10 + \text{last_digit}$ is the one line that does the whole job. Example: rev starts 0; last digit 5 -> $\text{rev}=5$; next last digit 5 -> $\text{rev}=55$; next 3 -> $\text{rev}=553$... and so on.

How it works

- 1. Save the last digit: $n \% 10$.
- 2. Append it: $\text{rev} = \text{rev} * 10 + \text{digit}$.
- 3. Remove it: $n = n / 10$.
- 4. Repeat until n becomes 0.

Code

```
public class ReverseNumber {
    public static void main(String[] args) {
        int n = 12355;
        int rev = 0;

        while (n > 0) {
            int last_digit = n % 10;
            rev = rev * 10 + last_digit;
            n = n / 10;
        }
        System.out.println(rev);
    }
}
```

TIME: $O(\log_{10} n)$ SPACE: $O(1)$

3. Palindrome Number

LeetCode #9

Problem (short): Check whether a number reads the same forwards and backwards. 1221 is a palindrome, 123 is not.

LeetCode 9 - Palindrome Number (Easy). [Solve on LeetCode](#)

Question (full): Given an integer x, return true if x is a palindrome, and false otherwise. An integer is a palindrome when it reads the same forward and backward. Examples: 121 -> true; -121 -> false (121- reads backwards); 10 -> false.

Idea in simple words

Reverse the whole number with the trick from the previous problem, then compare the reversed number with the original. Equal means palindrome.

Note the code copies n into a variable first, because n gets destroyed while reversing. It also takes Math.abs so the sign does not break the comparison.

How it works

1. Copy the original number before changing it.
2. Reverse n digit by digit.
3. If reversed equals original, print palindrome, otherwise not.

Code

```
public class Palindrome {
    public static void main(String[] args) {
        int n = Math.abs(-1221);
        int original = n;
        int rev = 0;
        while (n > 0) {
            int last_digit = n % 10;
            rev = rev * 10 + last_digit;
            n = n / 10;
        }

        if (rev == original) {
            System.out.println("it is palindrome");
            return;
        } else {
            System.out.println("not a palindrome");
        }
    }
}
```

TIME: $O(\log_{10} n)$ SPACE: $O(1)$

4. GCD - Euclidean Algorithm

LeetCode #1979

Problem (short): Find the greatest common divisor of two numbers. $\text{gcd}(9, 12) = 3$.

LeetCode 1979 - Find Greatest Common Divisor of Array (Easy). [Solve on LeetCode](#)

Question (full): LeetCode 1979: given an integer array `nums`, return the greatest common divisor of the smallest and the largest number in `nums`. Example: `nums = [2,5,6,9,10]` -> $\text{gcd}(2, 10) = 2$. The repo solves the core sub-problem - gcd of two numbers - which you call with the array's min and max.

Idea in simple words

The GCD of a and b is the same as the GCD of b and the remainder $a \% b$. Keep replacing (a, b) with $(b, a \% b)$; the moment b becomes 0, a is the answer.

The swap keeps the remainder and the old b . This shrinks the numbers fast, so the loop runs very few times even for huge numbers.

How it works

- 1. While b is not 0: remember b , set $b = a \% b$, then $a = \text{old } b$.
- 2. When the loop ends, a is the GCD.

Code

```
public class Euclidean {
    public static void main(String[] args) {

        int a = 9;
        int b = 12;

        while(b != 0) {

            int temp = b;

            b = a % b;

            a = temp;
        }

        System.out.println("GCD = " + a);
    }
}
```

TIME: $O(\log \min(a, b))$

SPACE: $O(1)$

PART 03

Bitwise Tricks

The fastest operations on Earth - XOR, shifts and AND

C
o
m
p
u
t
e
r
s

s
t
o
r
e

n
u
m
b
e
r
s

a
s

0
s

a
n
d

1
s
.

B
i
t
w
i
s
e

o
p
e
r
a
t
o
r
s

w
o
r
k

o
n

t
h
o
s
e

0
s

a
n
d

1
s

d
i
r
e
c
t
l
y

a
n
d

a
r
e

l
i
g
h
t
n
i
n
g

f
a
s
t
.

T
h
r
e
e

f
a
c
t
s

y
o
u

m
u
s
t

m
e
m
o
r
i
s
e
:

a
^

a

=

0

(

a

n

u

m

b

e

r

X

O

R

i

t

s

e

l

f

c

a

n

c

e

l

s

t

o

0

)

,

a

^

0

=

a

,

a
n
d

n

&

1

r
e
a
d
s

t
h
e

l
a
s
t

b
i
t
.

X
O
R

i
s

t
h
e

s
u
p
e
r
p
o
w

e
r

h
e
r
e

-

i
t

i
s

h
o
w

w
e

f
i
n
d

a

n
u
m
b
e
r

t
h
a
t

a
p
p
e
a
r
s

o

n
c
e
.

1. Odd or Even

GfG

Problem (short): Check if a number is odd or even.

Practice: GfG - Check whether a number is odd or even

Question (full): Given an integer n, determine whether it is odd or even without using the modulo (%) operator. Examples: 5 -> odd; 4 -> even.

Idea in simple words

In binary, the last bit is 0 for even numbers and 1 for odd numbers. $n \& 1$ picks out that bit.

5 in binary is 0101, and $0101 \& 0001 = 0001 = 1$, so 5 is odd. No division needed at all.

How it works

- 1. If $(n \& 1) == 0$ print even, otherwise print odd.

Code

```
import java.util.*;

public class Odd_even{
    public static void main(String[] args) {
        int a = 5;
        if ((a & 1) == 0) {
            System.out.println("even");
        } else {
            System.out.println("odd");
        }
        //    o/p 5->  0101
        //                0001
        //                -----
        //                0001 ->1 (odd)
    }
}
```

TIME: O(1)

SPACE: O(1)

2. Multiply / Divide by 2 with Shifts

GfG

Problem (short): Double a number or halve it without using * or /.

Practice: GfG - Shift operators

Question (full): Double a number n with $n \ll 1$ and halve it with $n \gg 1$. Examples: 5 (0101) $\ll 1 \rightarrow$ 10 (1010); 6 (0110) $\gg 1 \rightarrow$ 3 (0011).

Idea in simple words

Shifting bits left ($\ll$) makes the number twice as big, shifting right ($\gg$) halves it.

This is how processors multiply and divide by powers of two internally. The speedup only matters at a very low level, but it is a famous interview fact.

How it works

- 1. $n \ll 1$ doubles n.
- 2. $n \gg 1$ halves n (integer division).

Code

```
import java.util.*;
public class Multiply_by_two {
    public static void main(String[] args) {
        //left shipt
        int n = 5;
        System.out.println(n << 1);

        // o/p
        // 5-> 0101
        // left shift(<<)-> 1010 (10)
        // output->10
    }
}

import java.util.*;

public class Divide_by_two {
    public static void main(String[] args) {

        //right shipt
        int n = 6;
        System.out.println(n >> 1);

        // output
        // n=4-> 0100
        // Right shipt(>>)-> 0010 (2)
        // ans->2
    }
}
```

TIME: O(1)

SPACE: O(1)

3. Count Set Bits

LeetCode #191

Problem (short): Count how many 1-bits are inside a number. 11 in binary is 1011, which has three 1s.

LeetCode 191 - Number of 1 Bits (Easy). [Solve on LeetCode](#)

Question (full): Write a function that takes the binary representation of an unsigned integer and returns the number of '1' bits it has (also called the Hamming weight). Example: 11 (binary 1011) -> 3; 128 (10000000) -> 1.

Idea in simple words

Method 1 reads the last bit ($n \& 1$) and shifts right until the number is 0. Method 2 (Brian Kernighan) is smarter: $n \& (n-1)$ always removes exactly one 1-bit, so just count how many times you can do it.

Kernighan's trick is a must-know: it skips the 0-bits entirely, so it runs only as many times as there are set bits.

How it works

- 1. Method 1: $\text{count} += (n \& 1)$; $n \gg= 1$; repeat while $n > 0$.
- 2. Method 2: $n = n \& (n - 1)$; $\text{count}++$; repeat while $n > 0$.

Code

```
import java.util.*;

public class count_set_bits {
    public static void main(String[] args) {
        int n = 11;
        int count = 0;
        while (n > 0) {
            count += (n & 1);
            n >>= 1;
        }
        System.out.println(count);

        // Method 2: Brian Kernighan's Algorithm
        int n2 = 29;
        int count2 = 0;
        while (n2 > 0) {
            n2 = n2 & (n2 - 1);
            count2++;
        }
        System.out.println(count2);
    }
}
```

TIME: O(bits) / O(set bits)

SPACE: O(1)

4. Power of Two

LeetCode #231

Problem (short): Check whether a number is a power of 2, like 4 (100) or 8 (1000).

LeetCode 231 - Power of Two (Easy). [Solve on LeetCode](#)

Question (full): Given an integer n, return true if it is a power of two. An integer n is a power of two if there exists an integer x such that $n == 2^x$. Examples: 1 -> true; 16 -> true; 3 -> false; 0 -> false.

Idea in simple words

Every power of 2 has exactly one 1-bit. The expression $n \& (n-1)$ clears the lowest 1-bit, so for a power of 2 the result is 0.

4 & 3 = 100 & 011 = 000 = 0. For 6 (110), 6 & 5 = 110 & 101 = 100, not 0, so 6 is not a power of two. Remember to reject $n \leq 0$. The repo's code only has the check - finish it by printing a message.

How it works

- 1. If $n > 0$ and $(n \& (n - 1)) == 0$, then n is a power of 2.

Code

```
import java.util.*;

public class Power_of_two {
    public static void main(String[] args) {
        int n = 4;
        if ((n & (n - 1)) == 0) {
            System.out.println("");
        }
    }
}
```

TIME: O(1)

SPACE: O(1)

5. Appears Once (XOR Superpower)

LeetCode #136

Problem (short): In an array where every number appears exactly twice except one, find the number that appears once. [2, 2, 1, 1, 3] -> 3.

LeetCode 136 - Single Number (Easy). [Solve on LeetCode](#)

Question (full): Given a non-empty array of integers `nums`, every element appears exactly twice except for one. Find that single element. You must implement a solution with linear runtime and constant extra space.
Example: `[2,2,1]` -> 1; `[4,1,2,1,2]` -> 4.

Idea in simple words

XOR every number together. Pairs cancel out ($a \oplus a = 0$) and anything XOR 0 stays itself, so the only thing left is the unique number.

No HashMap needed, no sorting needed. This single loop is the whole solution - clean and $O(n)$. This is the most reused trick in this booklet.

How it works

- 1. Start with `xor = 0`.
- 2. For every number: `xor = xor ^ number`.
- 3. Print `xor` - it is the unique element.

Code

```
import java.util.*;
public class Apperars_once {
    public static void main(String[] args) {
        int arr[] = { 2, 2, 1,1,3 };
        int xor = 0;
        for (int num : arr) {
            xor ^= num;
        }
        System.out.println(xor);
    }
}
```

TIME: $O(n)$

SPACE: $O(1)$

6. Two Unique Numbers

LeetCode #260

Problem (short): Every number appears twice except two numbers, which appear once. Find both. In `[1,1,2,2,3,4,4,5]` the answer is 3 and 5.

LeetCode 260 - Single Number III (Medium). [Solve on LeetCode](#)

Question (full): Given an integer array `nums` where exactly two elements appear only once and all the other elements appear exactly twice, return the two elements that appear only once. Return them in any order.
Example: `[1,2,1,3,2,5]` -> `[3,5]`.

Idea in simple words

XOR all numbers to get $a \oplus b$ (the XOR of the two unique numbers). Since a and b differ, there is at least one bit where they differ. Take the rightmost such bit, then split the array into two groups by that bit - each group now contains exactly one unique number, so XOR each group separately.

The rightmost set bit is found with `xor & -xor`, a classic one-liner. Every pair lands in the same group (they share that bit), while a and b land in different groups.

How it works

- 1. XOR all -> `xor = a ^ b`.
- 2. Find the rightmost set bit: `bit = xor & -xor`.
- 3. Loop again: if `(num & bit) != 0` XOR into `num1`, else XOR into `num2`.
- 4. `num1` and `num2` are the answers.

Code

```
import java.util.*;
public class Two_unique_numbers {
    public static void main(String[] args) {
        int arr[] = { 1,1, 2, 2, 3, 4, 4,5 };
        int xor=0;
        for (int i = 0; i < arr.length; i++) {
            xor ^= arr[i];
        }

        int RightMostSetBit =xor & -xor;

        int num1 = 0, num2 = 0;
        for (int i = 0; i < arr.length; i++) {
            if ((arr[i] & RightMostSetBit) != 0) {
                num1 ^= arr[i];
            } else {
                num2 ^= arr[i];
            }
        }
        System.out.println(num1+" "+num2);
    }
}
```

TIME: $O(n)$ **SPACE: $O(1)$**

PART 04

Patterns

Nested loops made visible - the shapes behind every grid problem

P
a
t
t
e
r
n
s

a
r
e

j
u
s
t

t
w
o

l
o
o
p
s
:

t
h
e

o
u
t
e
r

l
o
o
p

p
i

c
k
s

t
h
e

r
o
w
,

t
h
e

i
n
n
e
r

l
o
o
p

p
r
i
n
t
s

o
n
e

r
o
w
,
s

c
o
n
t
e
n

t
s
.

l
f

t
h
e

i
n
n
e
r

l
o
o
p

g
o
e
s

u
p

t
o

i
,

y
o
u

g
e
t

a

g
r
o
w
i

n
g

t
r
i
a
n
g
l
e
;

i
f

i
t

g
o
e
s

d
o
w
n

f
r
o
m

n
,

y
o
u

g
e
t

a

s
h
r
i

n
k
i
n
g

o
n
e
.

O
n
c
e

y
o
u

c
a
n

m
e
n
t
a
l
l
y

p
r
e
d
i
c
t

a

s
h
a
p
e

f
r

o
m

i
t
s

l
o
o
p

c
o
n
d
i
t
i
o
n
s
,

g
r
i
d

p
r
o
b
l
e
m
s

b
e
c
o
m
e

e
a
s
y
.

1. Pattern Gallery (1-7)

GfG

Problem (short): Print seven classic shapes: full square, growing triangles of stars and numbers, shrinking triangles, and a pyramid.

Practice: GfG - Print patterns with nested loops

Question (full): Print star / number patterns using nested loops. Outer loop = rows, inner loop = columns. Practice target: produce each shape for any n without memorising - derive it from the loop bounds. Not a LeetCode problem; every DSA course uses these as loop warm-ups.

Idea in simple words

All seven share one structure - outer loop i for rows, inner loop j for columns. The difference is only in the inner loop's end condition ($\leq i$ for growing, $> i$ for shrinking) and what you print (star, j+1, or i).

Shape 3 prints j+1 so each row counts up 1..i. Shape 4 prints i so each row repeats its own row number. Shape 7 is a pyramid: n-i-1 spaces then 2i+1 stars. (In the repo's 7th pattern the left-space loop prints an empty string instead of a space, so the pyramid is left-flushed - fix it by printing a space there.)

How it works

- 1. Outer loop: i = 0 to n-1 (rows).
- 2. Inner loop: j = 0 to some bound based on i (columns).
- 3. Print the chosen character or number, then a newline after each row.

Code

Pattern 1 - solid 5x5 square

```
import java.util.*;
public class firstpattern {
    public static void main(String[] args){
        for (int i = 0; i < 5; i++) {
            for (int j = 0; j < 5; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

Pattern 2 - right triangle of stars

```
import java.util.*;
public class secoundpattern {
    public static void main(String[] args) {
        for (int i = 0; i < 5; i++) {
            for (int j = 0; j <= i; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

Pattern 3 - counting triangle 1..i

```
import java.util.*;

public class thirdpattern {
    public static void main(String[] args){
        for (int i = 0; i < 5; i++) {
            for (int j = 0; j <= i; j++) {
                System.out.print(j+1);
            }
            System.out.println();
        }
    }
}
```

Pattern 4 - row number repeated

```
import java.util.*;

public class fourthpattern {
    public static void main(String[] args) {
        for (int i = 0; i <=5; i++) {
            for (int j = 0; j < i; j++) {
                System.out.print(i);
            }
            System.out.println();
        }
    }
}
```

Pattern 5 - inverted triangle

```
import java.util.*;

public class fifthpatten {
    public static void main(String[] args) {
        for (int i = 0; i <= 5; i++) {
            for (int j = 5; j > i; j--) {
                System.out.print(" ");
            }
            System.out.println();
        }
    }
}
```

Pattern 6 - counting-down triangle

```
import java.util.*;

public class sixthpattern {
    public static void main(String[] args){
        for (int i = 0; i < 5; i++) {
            for (int j = 5; j > i; j--) {
                System.out.print(5-j+1);
            }
            System.out.println();
        }
    }
}
```

Pattern 7 - pyramid (left space loop prints empty - fix to a space)

```
import java.util.*;
public class seventhpattern {
    public static void main(String[] args) {
        int n=5;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                System.out.print("");
            }
            for (int j = 0; j < 2 * i + 1; j++) {
                System.out.print("*");
            }
            for (int j = 0; j < n - i - 1; j++) {
                System.out.print(" ");
            }
            System.out.println("");
        }
    }
}
```

TIME: $O(n^2)$ **SPACE: $O(1)$**

PART 05

Arrays

The heart of this repo - two pointers, sliding window, XOR and Kadane

A
r
r
a
y

p
r
o
b
l
e
m
s

a
r
e

u
s
u
a
l
l
y

w
o
n

b
y

a

t
e
c
h
n
i
q
u
e
,

n
o
t

r
a
w

e
f
f
o
r
t
.

T
h
e

r
e
c
u
r
r
i
n
g

w
e
a
p
o
n
s

h
e
r
e
:

t
w
o

p
o

i
n
t
e
r
s

m
e
e
t
i
n
g

i
n

t
h
e

m
i
d
d
l
e
,

a

s
l
o
w

a
n
d

f
a
s
t

p
o
i
n
t

e
r
,

X
O
R

f
o
r

u
n
i
q
u
e
n
e
s
s
,

a
n
d

r
u
n
n
i
n
g

s
u
m
s
.

M
o
s
t

g
o
o
d

s
o
l
u
t
i
o
n
s

s
c
a
n

t
h
e

a
r
r
a
y

o
n
c
e

-

t
h
a
t

i
s

O
(
n
)

a
n
d

h

a
r
d

t
o

b
e
a
t
.

1. Reverse an Array

GfG

Problem (short): Reverse the order of elements in place. [1, 2, 5, 6, 4] -> [4, 6, 5, 2, 1].

Practice: [GfG - Reverse an array](#)

Question (full): Given an array, reverse its elements in place. Example: [1,2,5,6,4] -> [4,6,5,2,1]. (The string version of this problem is LeetCode 344 Reverse String.)

Idea in simple words

Put one pointer at the start and one at the end. Swap their values, then move both towards the middle until they meet.

In-place means no extra array - we only swap. This two-pointer-meeting-in-the-middle shape shows up constantly.

How it works

- 1. left = 0, right = len-1.
- 2. While left < right: swap arr[left] and arr[right]; left++; right--.

Code

```
import java.util.Arrays;

public class Reversearray {
    public static void Reverse(int a[]) {
        int left = 0;
        int right = a.length - 1;
        while (left < right) {
            int temp = a[left];
            a[left] = a[right];
            a[right] = temp;
            left++;
            right--;
        }
        System.out.println(Arrays.toString(a));
    }
    public static void main(String[] args) {
        int a[] = { 1, 2, 5, 6, 4 };
        Reverse(a);
    }
}
```

TIME: O(n)

SPACE: O(1)

2. Check if the Array is Sorted

GfG

Problem (short): Say whether an array is sorted in non-decreasing order. [1,2,3,7,5] is not (7 > 5).

Practice: GfG - Check if array is sorted

Question (full): Given an array, check whether it is sorted in non-decreasing order. Examples: [1,2,3,7,5] -> false (7 > 5); [1,2,3,4] -> true.

Idea in simple words

Walk through the array. The moment a neighbour pair is out of order ($\text{arr}[i] > \text{arr}[i+1]$) it is not sorted.

The code prints false and returns immediately on the first violation - no point scanning the rest.

How it works

- 1. For $i = 0$ to $n-2$:
- 2. If $\text{arr}[i] > \text{arr}[i+1]$, print false and stop.
- 3. If we finish the loop, the array is sorted.

Code

```
import java.util.*;
public class Check_if_array_sorted {
    public static void main(String[] args) {
        int[] arr = { 1, 2, 3, 7, 5, 6 };

        for (int i = 0; i < arr.length - 1; i++) {
            if (arr[i] > arr[i + 1]) {
                System.out.println("false");
                return;
            }
        }
        return ;
    }
}
```

TIME: $O(n)$ **SPACE: $O(1)$** **3. Move Zeros to the End****LeetCode #283**

Problem (short): Push all zeros to the end while keeping the order of the non-zero numbers. [1,0,5,8,1,0,1] -> [1,5,8,1,1,0,0].

LeetCode 283 - Move Zeroes (Easy). [Solve on LeetCode](#)

Question (full): Given an integer array nums, move all 0s to the end while maintaining the relative order of the non-zero elements. Do this in-place without making a copy of the array. Example: [0,1,0,3,12] -> [1,3,12,0,0].

Idea in simple words

A slow pointer marks where the next non-zero should be placed. A fast pointer scans ahead; whenever it finds a non-zero, it swaps it to the slow position.

The fast pointer always moves, the slow one only moves when it receives a non-zero. Zeros naturally get pushed right.

How it works

- 1. left = 0, right = 0.
- 2. While right < n: if $\text{arr}[\text{right}] \neq 0$, swap $\text{arr}[\text{left}]$ with $\text{arr}[\text{right}]$ and left++.
- 3. Always right++.

Code

```
import java.util.*;
public class Move_zero {
    public static void main(String[] args) {
        int[] arr = { 1, 0, 5, 8, 1, 0, 1 };
        int left=0;
        int right = 0;
        int temp;
        while (right<arr.length) {
            if (arr[right] != 0) {
                // swap arr[left] and arr[right]
                temp = arr[left];
                arr[left] = arr[right];
                arr[right] = temp;
                left++;
            }
            right++;
        }
        for (int i = 0; i < arr.length; i++) {
            System.out.println(arr[i]);
        }
    }
}
```

TIME: O(n)

SPACE: O(1)

4. Segregate Zeros and Ones

GfG

Problem (short): Push all zeros to the left side and all ones to the right side. [1,0,1,0,1,0] -> [0,0,0,1,1,1].

Practice: [GfG - Segregate 0s and 1s](#)

Question (full): Given a binary array (only 0s and 1s), move all 0s to the left side and all 1s to the right side. The order inside each group does not matter. Example: [1,0,1,0,1,0] -> [0,0,0,1,1,1]. This is the two-colour version of LeetCode 75 Sort Colors.

Idea in simple words

left hunts for a misplaced 1 from the left, right hunts for a misplaced 0 from the right. When both are found, swap them.

Three cases: left has 1 and right has 0 -> swap and move both; left already has 0 -> move left; right already has 1 -> move right. This is the same idea as the Dutch flag in problem 16 but with only two colours.

How it works

- 1. left = 0, right = n-1.
- 2. While left < right:
- 3. If left has 1 and right has 0, swap them, left++, right--.
- 4. If left has 0, left++. If right has 1, right--.

Code

```
import java.util.*;
public class Move_zero_and_one {
    public static void main(String[] args) {
        int[] arr = { 1, 0, 1, 0, 1, 0 };
        int left = 0;
        int right = arr.length - 1;
        while (left < right) {
            if (arr[left] == 1 && arr[right] == 0) {
                int temp = arr[left];
                arr[left] = arr[right];
                arr[right] = temp;
                left++;
                right--;
            } else if (arr[left] == 0) {
                left++;
            } else if (arr[right] == 1) {
                right--;
            }
        }
        for (int i = 0; i < arr.length; i++) {
            System.out.println(arr[i]);
        }
    }
}
```

TIME: $O(n)$ SPACE: $O(1)$

5. Count Frequency of Each Element

GfG

Problem (short): Count how many times each element appears. [10,5,8,2,10,10,5,8] -> 10:3, 5:2, 8:2, 2:1.

Practice: GfG - Frequencies of array elements

Question (full): Given an array, count how many times each distinct value appears. Example: [10,5,8,2,10,10,5,8] -> 10:3, 5:2, 8:2, 2:1. The professional solution uses a HashMap and runs in $O(n)$.

Idea in simple words

For each element not yet counted, scan the rest of the array and count matches, marking them visited so they are not counted again.

This brute force is $O(n^2)$. The pro version uses a HashMap and is $O(n)$ - the collection chapter at the end shows how. Understanding both versions matters for interviews.

How it works

- 1. Keep a visited[] flag array.
- 2. For each unvisited i, count equal elements j > i and mark them visited.
- 3. Print element and its count.

Code

```
import java.util.*;
public class Count_frequency {
    public static void main(String[] args) {
        int[] arr = { 10, 5, 8, 2, 10, 10, 5, 8 };
        boolean[] visited = new boolean[arr.length];
        for (int i = 0; i < arr.length; i++) {
            if (visited[i]) {
                continue;
            }
            int count = 1;
            for (int j = i+1; j < arr.length; j++) {
                if (arr[i] == arr[j]) {
                    visited[j] = true;
                    count++;
                }
            }
            System.out.println(" "+arr[i]+" -> "+count);
        }
    }
}
```

TIME: $O(n^2)$ SPACE: $O(n)$

6. Palindrome Check (String)

LeetCode #125

Problem (short): Check whether a string reads the same forwards and backwards. 'madam' is a palindrome, 'hello' is not.

LeetCode 125 - Valid Palindrome (Easy). [Solve on LeetCode](#)

Question (full): Given a string s, return true if it is a palindrome. On LeetCode: first convert all uppercase letters to lowercase and remove all non-alphanumeric characters, then check if it reads the same forward and backward. Example: 'A man, a plan, a canal: Panama' -> true. The repo's version is the simpler case-sensitive check - the two-pointer core is identical.

Idea in simple words

Compare the first and last characters, then move inwards. Any mismatch means it is not a palindrome.

Same two-pointer shape as reversing an array, but here we only compare instead of swapping.

How it works

- 1. left = 0, right = len-1.
- 2. While left < right:
- 3. If char[left] != char[right], return false.
- 4. left++, right--. Return true when they cross.

Code

```
public class PalindromArray {

    public static boolean Pailndrome(String str) {
        int left = 0;
        int right = str.length() - 1;
        while (left < right) {
            if (str.charAt(left) != str.charAt(right)) {
                return false;
            }
            left++;
            right--;
        }
        return true;
    }

    public static void main(String[] args) {
        System.out.println(Pailndrome("madam"));
    }
}
```

TIME: O(n)

SPACE: O(1)

7. Two Sum in a Sorted Array

LeetCode #167

Problem (short): Find two numbers in a sorted array that add up to a target. [1,2,3,4,6] with target 8 -> 2 + 6.

LeetCode 167 - Two Sum II - Input Array Is Sorted (Medium). [Solve on LeetCode](#)

Question (full): Given a 1-indexed array of integers numbers that is already sorted in non-decreasing order, find two numbers that add up to a specific target. Return the 1-indexed positions as [index1, index2]. Example: numbers = [2,7,11,15], target = 9 -> [1,2].

Idea in simple words

Start with the smallest and largest numbers. If their sum is too small, the only way to grow it is to move the left pointer right; if too big, move the right pointer left.

Because the array is sorted, moving left increases the sum and moving right decreases it - so one pass is enough.

How it works

- 1. left = 0, right = n-1.
- 2. Compute sum = a[left] + a[right].
- 3. sum == target -> found. sum < target -> left++. Else right--.

Code

```
class Twopointer {
    public static void Pointer(int a[], int target) {
        int left = 0;
        int right = a.length - 1;
        while (left < right) {
            int sum = a[left] + a[right];
            if (sum == target) {
                System.out.println("sum found " + a[left] + " " + a[right]);
                return;
            } else if (sum < target) {
                left++;
            } else {
                right--;
            }
        }
        System.out.println("not found");
    }
    public static void main(String[] args) {
        int a[] = { 1, 2, 3, 4, 6 };
        Pointer(a, 8);
    }
}
```

TIME: O(n)

SPACE: O(1)

8. Max Sum of Subarray of Size k

LeetCode #643

Problem (short): Find the maximum sum of any window of k consecutive numbers. [1,2,3,4,5,6], k=3 -> 4+5+6 = 15.

LeetCode 643 - Maximum Average Subarray I (Easy). [Solve on LeetCode](#)

Question (full): Given an integer array nums and an integer k, find a contiguous subarray of length k with the maximum average value and return that average. Example: nums = [1,12,-5,-6,50,3], k = 4 -> 12.75. The repo computes the maximum sum of the window; the average is that sum divided by k.

Idea in simple words

Sum the first k numbers. Then slide the window: drop the element leaving on the left and add the one entering on the right. Keep the best sum.

We reuse the old sum instead of re-adding k numbers each time, turning $O(n*k)$ into $O(n)$. This is the classic sliding-window trick.

How it works

- 1. Sum the first k elements into sum.
- 2. For $i = k$ to $n-1$: $\text{sum} = \text{sum} - \text{arr}[i-k] + \text{arr}[i]$; $\text{max} = \max(\text{max}, \text{sum})$.
- 3. Print max.

Code

```
public class SlidingWindow {
    public static void main(String[] args) {
        int arr[] = { 1, 2, 3, 4, 5, 6 };
        int sum = 0;
        int k = 3;

        for (int i = 0; i < k; i++) {
            sum += arr[i];
        }

        int max = sum;
        for (int i = k; i < arr.length; i++) {
            sum = sum - arr[i - k] + arr[i];
            max = Math.max(max, sum);
        }
        System.out.println(max);
    }
}
```

TIME: $O(n)$

SPACE: $O(1)$

9. Left Rotate by One

GfG

Problem (short): Move every element one step left, with the first element wrapping around to the end.
 $[1,2,3,4,5,6] \rightarrow [2,3,4,5,6,1]$.

Practice: [GfG - Left rotate an array by one](#)

Question (full): Given an array, left rotate it by one position: every element shifts one place left and the first element wraps to the end. Example: $[1,2,3,4,5,6] \rightarrow [2,3,4,5,6,1]$. (For rotating by k positions, practice LeetCode 189 Rotate Array.)

Idea in simple words

Save the first element, shift everything else one position left, then put the saved element at the end.

For rotating by d positions, the standard trick is three reversals: reverse the whole array, reverse the first $n-d$, reverse the last d. Still $O(n)$.

How it works

- 1. Save $\text{arr}[0]$.
- 2. For $i = 1$ to $n-1$: $\text{arr}[i-1] = \text{arr}[i]$.
- 3. Put the saved value in $\text{arr}[n-1]$.

Code

```
import java.util.*;
class Left_Rotate{
    public static void main(String[] args){
        int[] arr={1,2,3,4,5,6};
        int initial=arr[0];
        for(int i=1;i<arr.length;i++){
            arr[i-1]=arr[i];
        }
        arr[arr.length-1]=initial;
        System.out.println(Arrays.toString(arr));
    }
}
```

TIME: O(n)

SPACE: O(1)

10. Second Smallest and Second Largest

GfG

Problem (short): Find the second smallest and second largest element of an array. [1,2,4,6,7,5] -> second smallest 2, second largest 6.

Practice: [GfG - Second largest / second smallest in array](#)

Question (full): Given an array, find the second smallest and second largest distinct values. Example: [1,2,4,6,7,5] -> second smallest 2, second largest 6. If there are fewer than 2 elements, output -1 -1. (Harder version: LeetCode 215 Kth Largest Element in an Array.)

Idea in simple words

First pass finds the smallest and largest. Second pass finds the smallest value that is not the smallest, and the largest value that is not the largest.

If the array has fewer than 2 elements, the answer is -1 -1. The code handles that edge case first. (This can be done in one pass with four running variables, a nice interview follow-up.)

How it works

- 1. Edge case: if $n < 2$ print -1 -1 and stop.
- 2. Pass 1: find smallest and largest.
- 3. Pass 2: find the closest values to them that are different.

Code

```

import java.util.*;
class Solution {

    // Method to find the second smallest and second largest elements in the array
    public static void getElements(int[] arr, int n) {

        // Edge case: when the array has less than 2 elements
        if (n < 2) {
            System.out.println(-1 + " " + -1); // Print -1 for both second smallest and second largest
            return;
        }

        // Initialize variables to track the smallest, second smallest, largest, and second largest elements
        int small = Integer.MAX_VALUE, second_small = Integer.MAX_VALUE;
        int large = Integer.MIN_VALUE, second_large = Integer.MIN_VALUE;

        // Find the smallest and largest elements in the array
        for (int i = 0; i < n; i++) {
            small = Math.min(small, arr[i]); // Update the smallest element
            large = Math.max(large, arr[i]); // Update the largest element
        }

        // Find the second smallest and second largest elements
        for (int i = 0; i < n; i++) {
            if (arr[i] < second_small && arr[i] != small) {
                second_small = arr[i]; // Update second smallest if a smaller element is found
            }
            if (arr[i] > second_large && arr[i] != large) {
                second_large = arr[i]; // Update second largest if a larger element is found
            }
        }

        // Output the second smallest and second largest elements
        System.out.println("Second smallest is " + second_small);
        System.out.println("Second largest is " + second_large);
    }

    public class Second_Smallest_and_Second_Largest_Element {
        public static void main(String[] args) {
            int n = 6;
            int[] arr = {1, 2, 4, 6, 7, 5}; // Array of elements

            // Call the function to find and print the second smallest and second largest elements

            // ... (continued) ...
            Solution.getElements(arr, n);
        }
    }
}

```

TIME: O(n)

SPACE: O(1)

11. Maximum Subarray Sum - Kadane's Algorithm

LeetCode #53

Problem (short): Find the subarray (a continuous slice) with the largest sum. [-2,1,-3,4,-1,2,1,-5,4] -> subarray [4,-1,2,1] = 6.

LeetCode 53 - Maximum Subarray (Medium). [Solve on LeetCode](#)

Question (full): Given an integer array nums, find the contiguous subarray (containing at least one number) which has the largest sum and return its sum. Example: [-2,1,-3,4,-1,2,1,-5,4] -> 6 (the subarray [4,-1,2,1] sums to 6).

Idea in simple words

The golden rule written in the code: if your running sum becomes negative, throw it away and start a new subarray. Always remember the biggest sum you have seen.

A negative running sum can only drag future sums down, so resetting it to 0 is always safe. One pass, no extra memory - this is a famous and elegant algorithm.

How it works

- 1. currentSum = 0, maxSum = very small.
- 2. For each num: currentSum += num; maxSum = max(maxSum, currentSum).
- 3. If currentSum < 0, set it to 0 and start over.

Code

```
import java.util.*;
public class MaxSubArray {
    public static void main(String[] args) {
        //Kadane's Algorithm

        // Golden Rule

        // If your running sum becomes negative,

        // throw it away.

        // Start a new subarray.

        // That's the entire algorithm.

        int arr[] = { -2, 1, -3, 4, -1, 2, 1, -5, 4 };
        int currentSum = 0;
        int maxSum = Integer.MIN_VALUE;

        for (int num : arr) {
            currentSum += num;
            maxSum = Math.max(maxSum, currentSum);
            if (currentSum < 0) {
                currentSum = 0;
            }
        }
        System.out.println(maxSum);
    }
}
```

TIME: O(n)**SPACE: O(1)**

12. Missing Number (Sum Method)

LeetCode #268

Problem (short): Given numbers 1..n with one missing, find it. [1,2,3,5] -> missing is 4.

LeetCode 268 - Missing Number (Easy). [Solve on LeetCode](#)

Question (full): Given an array nums containing n distinct numbers in the range [0, n], return the only number in the range that is missing from the array. Example: [3,0,1] -> 2. The repo solves the 1..n version with the sum formula; LeetCode 268 uses the range [0, n].

Idea in simple words

The sum of 1..n is always $n*(n+1)/2$. Subtract the actual sum of the array from this expected sum - the difference is the missing number.

The code uses the last element of the array as n, which only works because the array is sorted here. The XOR version (Basic Maths folder) avoids overflow and works in any order.

How it works

- 1. expected = $n*(n+1)/2$.
- 2. Sum the array into realSum.
- 3. Missing = expected - realSum.

Code

```
import java.util.*;

public class Missing_num {
    public static void main(String[] args) {
        int arr[] = { 1, 2, 3, 5 };
        int n = arr[arr.length - 1];
        int Excepted_sum = (n * (n + 1)) / 2;
        int Real_sum=0;
        for (int i = 0; i < arr.length; i++) {
            Real_sum += arr[i];
        }
        System.out.println(Excepted_sum-Real_sum);
    }
}
```

TIME: O(n)

SPACE: O(1)

13. Missing Number (XOR Method)

LeetCode #268

Problem (short): Same problem - find the missing number from 1..n, but with XOR so it works for huge n and any order. [1,2,4,5,3,7] -> 6.

LeetCode 268 - Missing Number (XOR solution) (Easy). [Solve on LeetCode](#)

Question (full): Same as LeetCode 268: given nums containing n distinct numbers in [0, n], return the missing one. The XOR solution never overflows, which is why many people prefer it over the sum formula. Example: [9,6,4,2,3,5,7,0,1] -> 8.

Idea in simple words

XOR every number from 1 to n, then XOR the result with every array element. Everything present cancels out ($a \oplus a = 0$), leaving only the missing number.

This is the cleanest way to spot a missing value and never overflows, unlike the sum formula.

How it works

- 1. xorAll = XOR of all integers 0..n.
- 2. xorArr = XOR of the array.
- 3. Answer = xorAll ^ xorArr.

Code

```
import java.util.*;

public class Missing_num {
    public static void main(String[] args) {
        int arr[] = { 1, 2, 4, 5 ,3,7};
        int n = arr[arr.length - 1];

        int xorall = 0;
        for (int i = 0; i <= n; i++) {
            xorall = xorall ^ i;
        }
        int xorarr = 0;
        for (int i = 0; i < arr.length; i++) {
            xorarr = xorarr ^ arr[i];
        }
        int missing = xorall ^ xorarr;
        System.out.println(missing);
    }
}
```

TIME: O(n)

SPACE: O(1)

14. Maximum Consecutive Ones

LeetCode #485

Problem (short): Find the longest run of 1s in a binary array. [1,1,0,1,0,1,1,1] -> 3.

LeetCode 485 - Max Consecutive Ones (Easy). [Solve on LeetCode](#)

Question (full): Given a binary array nums, return the maximum number of consecutive 1s in the array.

Example: [1,1,0,1,1,1] -> 3.

Idea in simple words

Count the current run of 1s. Any 0 resets the counter to 0. Keep the largest count seen.

Track the best answer separately so resetting the current streak never loses the record.

How it works

- 1. count = 0, max = 0.
- 2. For each element: 1 -> count++; 0 -> count = 0.
- 3. max = max(max, count).

Code

```
import java.util.*;

public class Count_Maximum_Consecutive_One {
    public static void main(String[] args) {
        int arr[] = { 1, 1, 0, 1, 0, 1, 1, 1 };
        int count = 0;
        int max=0;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == 1) {
                count++;

            } else {
                count = 0;
            }
            max = Math.max(count, max);
        }
        System.out.println(max);
    }
}
```

TIME: O(n)

SPACE: O(1)

15. Remove Duplicates from a Sorted Array

LeetCode #26

Problem (short): Remove duplicates in place from a sorted array, returning the new length. [1,1,2,2,2,3,3,4,4,5] -> length 5.

LeetCode 26 - Remove Duplicates from Sorted Array (Easy). [Solve on LeetCode](#)

Question (full): Given an integer array nums sorted in non-decreasing order, remove the duplicates in-place so that each unique element appears only once. Return the number of unique elements k; the first k positions of nums must hold them in the original order. Example: [1,1,2] -> 2 with nums = [1,2, _].

Idea in simple words

A slow pointer i marks the last kept unique value. A fast pointer j scans; whenever it sees a different value, it places it right after i.

In-place means the unique values are copied over the duplicates at the front of the array - no new array needed.

How it works

- 1. i = 0. Handle empty array first.
- 2. For j = 1 to n-1: if arr[j] != arr[i], i++ and arr[i] = arr[j].
- 3. Answer is i + 1.

Code

```

import java.util.*;
public class Remove_Duplicate {
    public static void main(String[] args) {
        int arr[] = { 1, 1, 2, 2, 2, 3, 3, 4, 4,5 };

        //brute force
        // HashSet<Integer> newset=new HashSet<>();
        // for (int i = 0; i < arr.length; i++) {
        //     newset.add(arr[i]);
        // }
        // System.out.println(newset);

        //optimal(method 1 using new arr)
        // int left=0;
        // int right=1;

        // int[] newarr = new int[arr.length];
        // int i=1;
        // newarr[0]=arr[0];
        // while (right < arr.length) {
        //     if (arr[left] == arr[right]) {
        //         right++;
        //     } else {
        //         newarr[i] = arr[right];
        //         i++;
        //         left = right;
        //         right++;
        //     }
        // }
        // System.out.println(i);
        // for (int j = 0; j < newarr.length; j++) {
        //     System.out.println(newarr[j]);
        // }

        //method 2(without using new arr)
        int i = 0;
        if (arr.length == 0)
            System.out.println(0);

        for (int j = 1; j < arr.length; j++) {
            if (arr[j] != arr[i]) {
                i++;
                arr[i] = arr[j];
            }
        }

        // ... (continued) ...
        }
        System.out.println(i+1);
    }
}

```

TIME: O(n)

SPACE: O(1)

16. Sort 0, 1, 2 - Dutch National Flag

LeetCode #75

Problem (short): Sort an array that only holds 0s, 1s and 2s, in one pass. [0,2,1,2,0,1] -> [0,0,1,1,2,2].

LeetCode 75 - Sort Colors (Medium). [Solve on LeetCode](#)

Question (full): Given an array nums with n objects coloured red (0), white (1) or blue (2), sort them in-place so that objects of the same colour are adjacent, in the order red, white, blue. Solve without using the library's sort function. Example: [2,0,2,1,1,0] -> [0,0,1,1,2,2].

Idea in simple words

Three pointers: start keeps the 0-region boundary, mid scans, end keeps the 2-region boundary. A 0 at mid is swapped to start, a 2 is swapped to end, a 1 just moves on.

Named after the Dutch flag (three colours). It sorts in one pass with no counting array and no sort call.

How it works

- 1. start = 0, mid = 0, end = n-1.
- 2. While mid <= end:
- 3. arr[mid]==0 -> swap with start, start++, mid++.
- 4. arr[mid]==1 -> mid++.
- 5. arr[mid]==2 -> swap with end, end--.

Code

```
import java.util.*;

public class sortZeroOneTwo {

    public static void main(String[] args) {
        //Dutch National Flag algo
        int[] nums = { 0, 2, 1, 2, 0, 1 };
        int start = 0;
        int mid = 0;
        int end = nums.length - 1;
        while (mid <= end) {
            if (nums[mid] == 0) {
                int temp = nums[mid];
                nums[mid] = nums[start];
                nums[start] = temp;
                start++;
                mid++;
            } else if (nums[mid] == 1) {
                mid++;
            } else {
                int temp = nums[mid];
                nums[mid] = nums[end];
                nums[end] = temp;
                end--;
            }
        }
        for (int i = 0; i < nums.length; i++) {
            System.out.println(nums[i]);
        }
    }
}
```

TIME: O(n)

SPACE: O(1)

17. Union of Two Sorted Arrays

GfG

Problem (short): List all distinct numbers present in either array, in sorted order. [1,2,3,4,5] and [2,3,4,4,5] -> [1,2,3,4,5].

Practice: GfG - Union of two sorted arrays

Question (full): Given two sorted arrays, find their union: every distinct value that appears in either array, in sorted order. Example: [1,2,3,4,5] and [2,3,4,4,5] -> [1,2,3,4,5]. (GfG problem; no direct LeetCode equivalent.)

Idea in simple words

Walk both arrays with two pointers, always taking the smaller value and only adding it if it differs from the last value added. When one array runs out, copy the rest.

This is the same idea as merging two sorted lists, plus a duplicate check against the last added element.

How it works

- 1. Compare arr1[i] and arr2[j]; add the smaller if it is new.
- 2. If equal, add once and advance both.
- 3. After one array ends, drain the other with the same duplicate check.

Code

```
import java.util.*;
public class Union_of_Two_Sorted_Arrays {
    public static void main(String[] args) {
        int arr1[] = { 1, 2, 3, 4, 5 };
        int arr2[] = { 2, 3, 4, 4, 5 };
        List<Integer> Union = new ArrayList<>();
        int n = 5;
        int m = 5;
        int i = 0;
        int j = 0;
        while (i < n && j < m) {
            if (arr1[i] < arr2[j]) {
                if (Union.isEmpty() || Union.get(Union.size() - 1) != arr1[i]) {
                    Union.add(arr1[i]);
                    i++;
                }
            } else if (arr2[j] < arr1[i]) {
                if (Union.isEmpty() || Union.get(Union.size() - 1) != arr2[j]) {
                    Union.add(arr2[j]);
                    j++;
                }
            } else {
                if (Union.isEmpty() || Union.get(Union.size() - 1) != arr1[i]) {
                    Union.add(arr1[i]);
                    i++;
                    j++;
                }
            }
        }
        while (i < n) {
            if (Union.isEmpty() || Union.get(Union.size() - 1) != arr1[i])
                Union.add(arr1[i]);
            i++;
        }
        // Append remaining elements from arr2
        while (j < m) {
            if (Union.isEmpty() || Union.get(Union.size() - 1) != arr2[j])
                Union.add(arr2[j]);
            j++;
        }
        System.out.println(Union);
    }
}

// ... (continued) ...
}
```

TIME: $O(n + m)$ SPACE: $O(n + m)$

18. Longest Subarray with Sum k

GfG

Problem (short): Find the longest subarray whose numbers add up to exactly k. [10,5,2,7,1,9], k=15 -> [5,2,7,1] length 4.

Practice: [GfG - Longest subarray with given sum K](#)

Question (full): Given an array of positive integers and a target k, find the length of the longest contiguous subarray whose sum equals k. Example: [10,5,2,7,1,9], k = 15 -> 4 (subarray [5,2,7,1]). For arrays that also contain negatives or zeros, use the prefix-sum HashMap approach (LeetCode 560 Subarray Sum Equals K).

Idea in simple words

Sliding window works here because all numbers are positive: grow the window by adding the right element; while the sum exceeds k, shrink from the left. When sum equals k, record the length.

Heads up: this file is a work in progress and only has an empty while loop. Complete it with the two-pointer sliding window: right adds, left removes while $\text{sum} > k$, track the best length when $\text{sum} == k$.

How it works

- 1. $\text{left} = 0$, $\text{sum} = 0$, $\text{best} = 0$.
- 2. For right from 0 to $n-1$: $\text{sum} += \text{arr}[\text{right}]$.
- 3. While $\text{sum} > k$: $\text{sum} -= \text{arr}[\text{left}]$; $\text{left}++$.
- 4. If $\text{sum} == k$: $\text{best} = \max(\text{best}, \text{right} - \text{left} + 1)$.

Code

```
import java.util.*;

public class Longest_subarray_given_sum_k {
    public static void main(String[] args) {
        int arr[] = { 10, 5, 2, 7, 1, 9 };
        int k = 15;
        int maxLen = 0;
        int left = 0;
        int right = 0;
        int sum = arr[0];
        while (right < arr.length) {

        }
    }
}
```

TIME: $O(n)$

SPACE: $O(1)$

PART 06

Binary Search on 1D Arrays

Halving the problem - the fastest search there is

B
i
n
a
r
y

s
e
a
r
c
h

o
n
l
y

w
o
r
k
s

o
n

a

s
o
r
t
e
d

a
r
r
a
y
.

C
o

m
p
a
r
e

t
h
e

m
i
d
d
l
e

e
l
e
m
e
n
t

w
i
t
h

y
o
u
r

t
a
r
g
e
t
:

i
f

i
t

m
a

t
c
h
e
s
,

d
o
n
e
;

i
f

t
h
e

t
a
r
g
e
t

i
s

b
i
g
g
e
r
,

t
h
r
o
w

a
w
a
y

t
h

e

l
e
f
t

h
a
l
f
;

i
f

s
m
a
l
l
e
r
,

t
h
r
o
w

a
w
a
y

t
h
e

r
i
g
h
t

h
a
l
f
.

E
a
c
h

s
t
e
p

h
a
l
v
e
s

t
h
e

s
e
a
r
c
h

s
p
a
c
e
,

s
o

1

m
i
l
l
i
o
n

i
t

e
m
s

n
e
e
d

o
n
l
y

~
2
0

s
t
e
p
s
.

1. Binary Search

LeetCode #704

Problem (short): Search a sorted array for a target. [2,4,6,8,10,12,14] looking for 12 -> found.

LeetCode 704 - Binary Search (Easy). [Solve on LeetCode](#)

Question (full): Given an array of integers nums sorted in ascending order, and an integer target, write a function to search target in nums. If target exists, return its index. Otherwise, return -1. Must run in $O(\log n)$.
Example: nums = [-1,0,3,5,9,12], target = 9 -> 4.

Idea in simple words

Keep two pointers for the search range. Look at the middle. Adjust one pointer to discard the half that cannot contain the target.

$\text{mid} = \text{left} + (\text{right} - \text{left}) / 2$ is the safe way to compute the middle - it avoids integer overflow that $(\text{left} + \text{right}) / 2$ can hit. The loop continues while $\text{left} \leq \text{right}$.

How it works

- 1. $\text{left} = 0$, $\text{right} = n-1$.
- 2. While $\text{left} \leq \text{right}$: $\text{mid} = \text{left} + (\text{right}-\text{left})/2$.
- 3. $\text{arr}[\text{mid}] == \text{target} \rightarrow \text{found}$. $\text{arr}[\text{mid}] < \text{target} \rightarrow \text{left} = \text{mid}+1$. Else $\text{right} = \text{mid}-1$.
- 4. If the loop ends, target is not present.

Code

```
import java.util.*;

public class Binary_search {
    public static void main(String[] args) {
        int[] arr = { 2, 4, 6, 8, 10, 12, 14 };
        int left = 0;
        int right = arr.length - 1;
        int target = 12;
        int flag = 0;
        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (arr[mid] == target) {
                System.out.println("found");

                flag = 1;
                break;
            } else if (arr[mid] < target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        if (flag==0) {
            System.out.println("not found");
        }
    }
}
```

TIME: $O(\log n)$ SPACE: $O(1)$

2. Lower Bound

GfG

Problem (short): In a sorted array, find the first index where the value is $\geq$ a target. For [3,5,8,15,19] and $x=9$, the answer is index 3 (value 15).

Practice: [GfG - Lower bound in a sorted array](#)

Question (full): In a sorted array, find the first index i such that $arr[i] \geq x$. Example: [3,5,8,15,19], $x = 9 \rightarrow 3$. This is the exact idea behind LeetCode 35 Search Insert Position.

Idea in simple words

Run binary search but do not stop on a match. Whenever $arr[mid] \geq x$, record mid as a candidate and keep searching left for an earlier one.

This is the foundation of dozens of advanced problems (search insert position, first occurrence). The ans starts at $arr.length$, meaning 'not found' - it would go at the very end'.

How it works

- 1. $low = 0$, $high = n-1$, $ans = n$.
- 2. While $low \leq high$: $mid = low + (high-low)/2$.
- 3. If $arr[mid] \geq x$: $ans = mid$; $high = mid-1$. Else $low = mid+1$.
- 4. Print ans .

Code

```
import java.util.*;
public class LowerBoundFinder {
    public static void main(String[] args) {
        int[] arr = {3, 5, 8, 15, 19}; // Sorted array
        int x = 9; // Target value
        int low = 0; //start index
        int high = arr.length - 1; //end index
        int ans = arr.length; //default value if not found

        while (low <= high) {
            int mid = low + (high-low) / 2;

            if (arr[mid] >= x) {
                ans = mid;
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        }
        System.out.println(ans);
    }
}
```

TIME: $O(\log n)$ SPACE: $O(1)$

3. Upper Bound

GfG

Problem (short): In a sorted array, find the first index where the value is strictly greater than x. For [3,5,9,9,15,19] and x=9, the answer is index 4 (value 15).

Practice: GfG - Upper bound in a sorted array

Question (full): In a sorted array, find the first index i such that $\text{arr}[i] > x$. Example: [3,5,9,9,15,19], $x = 9 \rightarrow 4$. Lower and upper bounds together count occurrences of x - the idea behind LeetCode 34 Find First and Last Position.

Idea in simple words

Identical to lower bound but with a strict $>$ instead of $\geq$.

Lower bound and upper bound together give the count of a value: upper - lower tells you how many times x appears. Same $O(\log n)$ shape.

How it works

- 1. $\text{low} = 0$, $\text{high} = n-1$, $\text{ans} = n$.
- 2. While $\text{low} \leq \text{high}$: $\text{mid} = \text{low} + (\text{high}-\text{low})/2$.
- 3. If $\text{arr}[\text{mid}] > x$: $\text{ans} = \text{mid}$; $\text{high} = \text{mid}-1$. Else $\text{low} = \text{mid}+1$.
- 4. Print ans.

Code

```
import java.util.*;

public class UpperBound {
    public static void main(String[] args) {
        int[] arr = {3, 5, 9, 9, 15, 19}; // Sorted array
        int x = 9; // Target value
        int low = 0; //start index
        int high = arr.length - 1; //end index
        int ans = arr.length; //default value if not found

        while (low <= high) {
            int mid = low + (high-low) / 2;

            if (arr[mid] > x) {
                ans = mid;
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        }
        System.out.println(ans);
    }
}
```

TIME: $O(\log n)$ **SPACE: $O(1)$**

PART 07

Sorting

Three simple sorts - same $O(n^2)$ heart, different personalities

A
l
l

t
h
r
e
e

s
o
r
t
s

h
e
r
e

a
r
e

t
h
e

b
a
s
i
c
,

e
a
s
y
-
t
o
-
r
e

m
e
m
b
e
r

o
n
e
s
.

T
h
e
y

a
r
e

s
l
o
w

f
o
r

b
i
g

a
r
r
a
y
s

(
O
(
n
^
2
)
)

b
u
t

p
e
r
f
e
c
t

f
o
r

l
e
a
r
n
i
n
g

a
n
d

f
o
r

t
i
n
y

i
n
p
u
t
s
.

F
o
r

r

e
a
l

w
o
r
k

J
a
v
a

g
i
v
e
s

y
o
u

A
r
r
a
y
s
.
s
o
r
t
,

w
h
i
c
h

u
s
e
s

a

f

a
s
t

O
(
n

I
o
g

n
)

a
I
g
o
r
i
t
h
m

.

1. Bubble Sort

GfG

Problem (short): Sort an array. [52,55,8,5,2,1] -> [1,2,5,8,52,55].

Practice: [GfG - Bubble sort](#) ([practice: LC 912 Sort an Array](#))

Question (full): Sort an array by repeatedly swapping adjacent out-of-order pairs until no swaps are needed. Example: [52,55,8,5,2,1] -> [1,2,5,8,52,55]. Practice target: LeetCode 912 Sort an Array - but it expects $O(n \log n)$, so these basic sorts are for learning, not for submission.

Idea in simple words

Repeatedly walk the array and swap any two neighbours that are in the wrong order. After one full pass, the largest value has 'bubbled' to the end, so next pass can stop one position earlier.

The code adds an optimization: if a full pass makes no swaps, the array is already sorted, so it breaks out early. That makes the best case $O(n)$.

How it works

- 1. For $i = 0$ to $n-2$: assume no swaps (track = 0).
- 2. For $j = 0$ to $n-i-2$: if $arr[j] > arr[j+1]$ swap them and set track = 1.
- 3. If track is still 0, stop - the array is sorted.

Code

```
import java.util.*;
public class BubbleSort {
    public static void main(String[] args) {
        int arr[] = { 52, 55, 8, 5, 2, 1 };
        int n = arr.length;

        for (int i = 0; i < n - 1; i++) {
            int track=0;
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                    track = 1;
                }
            }
            if (track!=1) {
                break;
            }
        }
        for (int i = 0; i < n; i++) {
            System.out.println(arr[i]);
        }
    }
}
```

TIME: $O(n^2)$ worst / $O(n)$ best**SPACE: $O(1)$**

2. Selection Sort

GfG

Problem (short): Sort an array by repeatedly choosing the smallest remaining element.

Practice: GfG - Selection sort (practice: LC 912 Sort an Array)

Question (full): Sort an array by repeatedly finding the minimum of the unsorted part and swapping it to the front. Example: [25,5,32,2,1] -> [1,2,5,25,32].

Idea in simple words

Each pass finds the minimum of the unsorted part and swaps it into the first unsorted position. The sorted part grows from the left.

Selection sort always does the same number of comparisons regardless of the input - no best case, always $O(n^2)$. But it makes few swaps (at most n), which matters when swapping is expensive.

How it works

- 1. For $i = 0$ to $n-2$: assume $\text{min_index} = i$.
- 2. Scan $j = i+1$ to $n-1$; if $\text{arr}[j] < \text{arr}[\text{min_index}]$, update min_index .
- 3. Swap $\text{arr}[i]$ and $\text{arr}[\text{min_index}]$.

Code

```
import java.util.*;

public class SelectionSort {
    public static void main(String[] args) {
        int arr[] = { 25, 5, 32, 2, 1 };
        int n = arr.length;
        for (int i = 0; i < (n - 1); i++) {
            int min_index = i;
            for (int j = i + 1; j < n; j++) {
                if (arr[j] < arr[min_index]) {
                    min_index = j;
                }
            }
            int temp = arr[min_index];
            arr[min_index] = arr[i];
            arr[i] = temp;
        }
        for (int i = 0; i < arr.length; i++) {
            System.out.println(arr[i]);
        }
    }
}
```

TIME: $O(n^2)$ always**SPACE:** $O(1)$

3. Insertion Sort

GfG

Problem (short): Sort an array by inserting each element into its correct place among the already-sorted elements.

Practice: [GfG - Insertion sort \(linked-list version: LC 147\)](#)

Question (full): Sort an array by taking each element and inserting it into its correct place among the already-sorted elements on its left. Example: [25,3,5,88,1,5] -> [1,3,5,5,25,88]. (Linked-list version: LeetCode 147 Insertion Sort List.)

Idea in simple words

Imagine sorting playing cards: take the next card (the key), slide it left past all larger cards, and drop it into place. The left part of the array stays sorted at all times.

Insertion sort is great on almost-sorted data: if the input is already sorted it just checks each key once, so the best case is $O(n)$. It is also stable and used by Java internally for tiny arrays.

How it works

- 1. For $i = 1$ to $n-1$: $key = arr[i]$; $j = i-1$.
- 2. While $j \geq 0$ and $arr[j] > key$: shift $arr[j]$ right ($arr[j+1] = arr[j]$); $j--$.
- 3. Place the key at $arr[j+1]$.

Code

```
import java.util.*;
public class InsertionSort {
    public static void main(String[] args) {
        int arr[] = { 25, 3, 5, 88, 1, 5 };
        int n = arr.length;
        for (int i = 1; i < n; i++) {
            int key = arr[i];
            int j = i - 1;
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = key;
        }
        for (int i = 0; i < n; i++) {
            System.out.println(arr[i]);
        }
    }
}
```

TIME: $O(n^2)$ worst / $O(n)$ best**SPACE: $O(1)$**

PART 08

Strings

Strings are just character arrays - same tricks, different syntax

I
n

J
a
v
a

a

S
t
r
i
n
g

i
s

i
m
m
u
t
a
b
l
e
:

y
o
u

c
a
n
n
o
t

c
h
a
n

g
e

a

S
t
r
i
n
g
,

y
o
u

c
a
n

o
n
l
y

b
u
i
l
d

a

n
e
w

o
n
e
.

T
h
a
t

i
s

w
h
y

t
h
e

s
o
l
u
t
i
o
n
s

u
s
e

a

c
h
a
r
[
]

o
r

a

S
t
r
i
n
g
B
u
f
f
e
r
.

T

w
o
-
p
o
i
n
t
e
r

l
o
g
i
c

w
o
r
k
s

o
n

s
t
r
i
n
g
s

e
x
a
c
t
l
y

a
s

i
t

d
o
e

s
o
n

a
r
r
a
y
s
.

1. Reverse a String

LeetCode #344

Problem (short): Reverse the characters of a string. 'Rohann' -> 'nnahoR'.

LeetCode 344 - Reverse String (Easy). [Solve on LeetCode](#)

Question (full): Write a function that reverses a string. The input is given as a character array s. You must do it by modifying the input array in-place with $O(1)$ extra memory. Example: ['h','e','l','l','o'] -> ['o','l','l','e','h'].

Idea in simple words

Brute force: walk the string from the last character to the first, appending each one to a StringBuffer. Faster: StringBuffer already has a reverse() method.

The code shows both. The loop version teaches the logic; reverse() is what you would use in production.

How it works

- 1. Loop i from len-1 down to 0: append charAt(i).
- 2. Print the built string. Or just call new StringBuffer(s).reverse().

Code

```
import java.util.*;

public class Reverse_String {
    public static void main(String[] args) {

        //bruteforce
        String name = "Rohann";
        StringBuffer newName = new StringBuffer();
        for (int i = name.length() - 1; i >= 0; i--) {
            newName.append(name.charAt(i));
        }

        System.out.println(newName.toString());

        String a = "Hell0";
        StringBuffer b = new StringBuffer(a);

        System.out.println(b.reverse());
    }
}
```

TIME: $O(n)$

SPACE: $O(n)$

2. Reverse the Vowels

LeetCode #345

Problem (short): Reverse only the vowels in a string, leaving consonants in place. 'APPEIOU' -> the vowel order flips.

LeetCode 345 - Reverse Vowels of a String (Easy). [Solve on LeetCode](#)

Question (full): Given a string s, reverse only all the vowels in the string and return it. The vowels are a, e, i, o, u (and their uppercase forms). Example: 'IceCreAm' -> 'AceCreIm'; 'leetcode' -> 'leotcede'.

Idea in simple words

Two pointers, one from each end. Skip consonants on both sides; when both pointers rest on a vowel, swap them and move inwards.

Heads up: this file is incomplete - it has a syntax error at `a[left]=`. Finish it: check each side with a helper `isVowel`, skip non-vowels, and swap the two vowels when found. This is a classic interview warm-up.

How it works

- 1. `left = 0`, `right = len-1`.
- 2. Move left right until it hits a vowel; move right left until it hits a vowel.
- 3. Swap the two vowels, `left++`, `right--`. Repeat until they cross.

Code

```
import java.util.*;

public class Reverse_vowels {
    public static void main(String[] args) {
        String vowels = "APPEIOU";
        char[] a = vowels.toCharArray();
        int left = 0;
        int right = vowels.length() - 1;
        while (right > left) {
            if(a[left]=)
        }
    }
}
```

TIME: $O(n)$

SPACE: $O(n)$

PART 09

Java Collections

The built-in tools that replace raw arrays in real life

J
a
v
a

s
h
i
p
s

r
e
a
d
y
-
m
a
d
e

d
a
t
a

s
t
r
u
c
t
u
r
e
s
.

W
h
e
n

y
o

u

n
e
e
d

a

g
r
o
w
a
b
l
e

l
i
s
t
,

a

u
n
i
q
u
e
-
v
a
l
u
e

b
a
g
,

o
r

k
e
y
-

v
a
l
u
e

l
o
o
k
u
p
,

u
s
e

t
h
e
s
e

i
n
s
t
e
a
d

o
f

h
a
n
d
-
r
o
l
l
i
n
g

a
r
r

a
y
s
.

T
h
e

c
o
m
m
e
n
t
s

i
n
s
i
d
e

t
h
e

r
e
p
o

f
i
l
e
s

a
r
e

s
t
u
d
y

n

o
t
e
s

i
n

t
h
e
m
s
e
l
v
e
s

-

t
h
e

k
e
y

n
u
m
b
e
r
s

a
r
e

m
e
m
o
r
i
s
e
d

b
e
l
o
w
.

1. List Interface - ArrayList and LinkedList

GfG

Problem (short): Store an ordered collection that allows duplicates and can grow automatically.

Practice: GfG - Java collections (API knowledge, no LC)

Question (full): No LeetCode problem - this is core Java API knowledge. Know: ArrayList is a resizable array with $O(1)$ index access; LinkedList is a chain of nodes with $O(1)$ add/remove at the ends. The repo code demos add, get, set, remove, size, contains, isEmpty, addAll, toArray, sort, addFirst, addLast.

Idea in simple words

ArrayList is a resizable array: instant access by index ($O(1)$) but adding in the middle shifts everything ($O(n)$).

LinkedList is a chain of nodes: fast add/remove at the ends but slow random access.

Rule of thumb: read a lot -> ArrayList. Insert/delete at the front a lot -> LinkedList.

How it works

- 1. new ArrayList<>() gives a dynamic array with default capacity 10.
- 2. add appends ($O(1)$ amortized), get(i) reads instantly, remove(i) shifts ($O(n)$).
- 3. LinkedList addFirst/addLast insert at the ends in $O(1)$.

Code

```
import java.util.*;

public class ListInterface {
    public static void main(String[] args) {
        //arraylist
        //      ArrayList - Short Notes
        // 1. Definition
        // Part of java.util package.

        // Implements List interface.

        // Dynamic array -> grows/shrinks automatically.

        // 2. Key Features
        // Ordered collection -> maintains insertion order.

        // Allows duplicates.

        // Index-based access (like arrays).

        // Resizable -> unlike fixed-size arrays.

        // 3. Performance
        // Access (get/set) -> O(1) (fast).

        // Insert/Delete at end -> O(1) amortized.

        // Insert/Delete in middle -> O(n) (shifting required).

        // 4. Constructors
        // ArrayList() -> default capacity (10).

        // ArrayList(int capacity) -> custom initial capacity.

        // ArrayList(Collection c) -> copies another collection.

        // 5. Common Methods
        // add(E e) -> add element.

        // get(int index) -> retrieve element.

        // set(int index, E e) -> replace element.

        // remove(int index) / remove(Object o) -> delete.
```

```
// ... (continued) ...

// size() -> number of elements.

// clear() -> remove all elements.

// contains(Object o) -> check existence.

// isEmpty() -> check if empty.
    ArrayList<Integer> list = new ArrayList<>();
    ArrayList<Integer> list2 = new ArrayList<>();
    list.add(2);

    list.add(3);
    list.add(3);
    list2.add(4);
    list2.add(5);
    list2.add(6);
    System.out.println(list);
    System.out.println(list.size());
    list.remove(2);
    System.out.println(list);
    System.out.println(list.isEmpty());
    System.out.println(list.contains(2));
    list.addAll(list2);
    System.out.println(list);
    System.out.println(list.contains(4));
    System.out.println(list.get(2));
    Object[] obj = list.toArray();
    for (Object arr : obj) {
        System.out.println(arr);
    }
    list.sort(null );
    System.out.println(list);

    //linked list
    LinkedList<Integer> linkList = new LinkedList<>();
    linkList.addFirst(5);
    linkList.addFirst(6);
    linkList.addLast(10);
    System.out.println(linkList);

}
}
```

TIME: ArrayList: get O(1), add/remove mid O(n)

SPACE: O(n)

2. Set Interface - HashSet and TreeSet

GfG

Problem (short): Store a collection that never keeps duplicates.

Practice: GfG - Java collections (API knowledge, no LC)

Question (full): No LeetCode problem - core Java API knowledge. Know: a Set refuses duplicates; HashSet gives average O(1) add/remove/contains but no order; TreeSet keeps elements sorted at O(log n) per operation. The code also demos retainAll and containsAll.

Idea in simple words

A Set refuses the second copy: adding 5 twice keeps one 5. HashSet stores by hash so add/remove/contains are average O(1) but order is random. TreeSet keeps elements sorted at a cost of O(log n) per operation.

retainAll keeps only the elements both sets share; containsAll asks if one set holds every element of another.

How it works

- 1. setA.add(5) twice -> the set holds one 5.
- 2. retainAll(setB) keeps the common elements.
- 3. TreeSet prints sorted order; HashSet prints some arbitrary order.

Code

```
import java.util.*;
public class SetInterface {
    public static void main(String[] args) {
        //set using HashSet

        // 2. Key Features
        //     Unordered -> does not maintain insertion order.

        //     Allows one null element.

        //     No duplicate values.

        //     Fast operations (add, remove, search) -> average O(1).

        Set<Integer> setA = new HashSet<>();
        Set<Integer> setB = new HashSet<>();
        setA.add(5);
        setA.add(5);
        System.out.println(setA); // prints a single 5; sets store unique elements and HashSet order is not guaranteed
        setA.add(6);
        setB.add(6);
        setB.add(7);
        setA.retainAll(setB);
        System.out.println(setA);
        System.out.println(setA.containsAll(setB)); // false
        HashSet<String> set = new HashSet<>();
        set.add("Java");
        set.add("Python");
        set.add("Java"); // duplicate ignored

        System.out.println(set); // [Java, Python] (order not guaranteed)
        System.out.println(set.contains("Python")); // true
        set.remove("Java");
        System.out.println(set); // [Python]

        //TreeSet will store in sorted orderopen

        Set<Integer> treeSet = new TreeSet<>();
        treeSet.add(55);
        treeSet.add(65);
        treeSet.add(2);
        System.out.println(treeSet);
    }
}
```

TIME: HashSet O(1) avg, TreeSet O(log n)

SPACE: O(n)

3. Map Interface - HashMap

GfG

Problem (short): Store values keyed by unique keys, like a phone book: name -> number.

Practice: GfG - Java collections (API knowledge, no LC)

Question (full): No LeetCode problem - core Java API knowledge, but HashMaps appear in many LC solutions (e.g. Two Sum, Top K Frequent). Know: put/get are average O(1); keys are unique - re-putting overwrites. The code demos put, get, putAll, remove, size, putIfAbsent, getOrDefault, containsValue, entrySet.

Idea in simple words

A HashMap maps each key to a value using hashing, so put and get are average O(1). Keys are unique - putting the same key again overwrites the old value.

putIfAbsent adds only if the key is missing; getOrDefault returns a default instead of null; entrySet gives all key=value pairs for looping.

How it works

- 1. put(key, value) inserts or overwrites.

- 2. get(key) reads the value; getOrDefault(key, default) avoids null.
- 3. entrySet() gives all key=value pairs for looping.

Code

```
import java.util.*;
public class MapInterface {
    public static void main(String[] args) {
        Map<Integer, String> m = new HashMap<>();

        //insertion
        m.put(1, "rohan");
        m.put(2, "aditya");

        //get method
        System.out.println(m.get(2)); //aditya

        Map<Integer, String> table = new HashMap<>();
        table.put(3, "ash");
        table.putAll(m);
        System.out.println(table); //{1=rohan, 2=aditya, 3=ash}

        //deletion
        table.remove(3);
        System.out.println(table);

        //size
        System.out.println(table.size()); //2

        //put if absent
        table.putIfAbsent(4, "roy");
        System.out.println(table); //{1=rohan, 2=aditya, 4=roy}

        //getOrDefault
        System.out.println(table.getOrDefault(5, "none")); //none

        System.out.println(table.getOrDefault(4, "none")); //roy

        //containsValue()
        System.out.println(table.containsValue("roy")); //true

        System.out.println(table.entrySet()); //[1=rohan, 2=aditya, 4=roy]

    }
}
```

TIME: O(1) average per operation

SPACE: O(n)

PART 10

Cheat Sheet

Every problem at a glance - technique, time, space, LeetCode number

Problem / Algorithm	Key Technique	Time	Space	LeetCode
Count digits / reverse / palindrome	Modulo + divide by 10	$O(\log n)$	$O(1)$	LC 7, 9
GCD (Euclid)	Remainder recursion	$O(\log \min)$	$O(1)$	LC 1979
Odd/even, shift, power of 2	Bitwise	$O(1)$	$O(1)$	LC 231
Count set bits	$n \& (n-1)$	$O(\text{set bits})$	$O(1)$	LC 191
Appears once / missing number	XOR cancel	$O(n)$	$O(1)$	LC 136, 268
Two unique numbers	XOR + partition bit	$O(n)$	$O(1)$	LC 260
Patterns	Nested loops	$O(n^2)$	$O(1)$	GfG
Reverse array / palindrome	Two pointers	$O(n)$	$O(1)$	LC 344, 125
Sorted check / max consecutive ones	Single scan	$O(n)$	$O(1)$	LC 485
Move zeros / segregate 0-1	Slow-fast / left-right	$O(n)$	$O(1)$	LC 283
Count frequency	visited array	$O(n^2)$	$O(n)$	GfG
Two sum sorted	Two pointers	$O(n)$	$O(1)$	LC 167
Max sum window k	Sliding window	$O(n)$	$O(1)$	LC 643
Max subarray	Kadane	$O(n)$	$O(1)$	LC 53
Remove duplicates sorted	Slow-fast	$O(n)$	$O(1)$	LC 26
Sort 0,1,2	Dutch flag	$O(n)$	$O(1)$	LC 75
Union of sorted arrays	Merge	$O(n+m)$	$O(n+m)$	GfG
Longest subarray sum k	Sliding window	$O(n)$	$O(1)$	LC 560
Binary search / lower / upper bound	Halving	$O(\log n)$	$O(1)$	LC 704, 35, 34
Bubble sort	Swap neighbours	$O(n^2)$	$O(1)$	LC 912
Selection sort	Pick minimum	$O(n^2)$	$O(1)$	LC 912
Insertion sort	Insert like cards	$O(n^2)$	$O(1)$	LC 147
Reverse string / vowels	Loop / two pointers	$O(n)$	$O(n)$	LC 344, 345
ArrayList	Dynamic array	get $O(1)$	$O(n)$	Java API
HashSet	Hash table	$O(1)$ avg	$O(n)$	Java API
HashMap	Hash table	$O(1)$ avg	$O(n)$	Java API

Final tip: for every problem here, first say out loud the technique (two pointers, sliding window, XOR, halving), then the time complexity, then its LeetCode number. If you can do all three from memory, you are ready for the interview room.